

Relatório — Global Solution 2026

Sistema de Monitoramento de Missão Espacial

Projeto: Global Solution 2026 — Monitoramento de Missão Espacial

Repositório: <https://github.com/FIAP-workd/global-solutions-1.1>

Equipe: Gabriel Coutinho Barcelos, Pedro César Fernandes de Brito, Luis Gustavo Andrade, KAIO ABREU BRIEGAS, João Victor Feitosa

1. Introdução

O projeto desenvolvido tem como objetivo simular um sistema de monitoramento para uma missão espacial experimental. A proposta é acompanhar dados de telemetria da colônia, analisar o estado dos módulos críticos, identificar falhas, gerar alertas e apresentar recomendações automáticas para manter a missão em funcionamento.

O sistema foi construído em Python e organizado de forma modular, separando as responsabilidades em arquivos diferentes. Essa divisão facilita a leitura do código e deixa mais claro o papel de cada parte do sistema, como geração dos dados, leitura da telemetria, diagnóstico operacional, alertas, previsão e recomendações.

A missão simulada recebe o nome de **Ares Habitat Experimental**. A partir das leituras horárias, o programa consegue avaliar energia, ambiente, comunicação, suporte à vida, eventos operacionais e inconsistências nos dados.

2. Objetivo do sistema

O objetivo principal do sistema é transformar dados simulados de uma missão espacial em informações úteis para tomada de decisão. Em vez de apenas exibir números, o programa interpreta os dados e classifica a operação da missão em três estados:

- **NORMAL:** quando os sistemas estão dentro dos limites esperados;
- **ALERTA:** quando existe algum risco ou sinal de instabilidade;
- **CRÍTICO:** quando há falha grave ou risco direto para a missão.

Além da classificação geral, o sistema também gera alertas automáticos, identifica inconsistências, calcula previsões simples e recomenda ações operacionais. Dessa forma, ele representa uma versão simplificada de uma central de monitoramento de uma colônia espacial.

3. Estrutura geral do projeto

O projeto foi organizado em módulos para separar melhor as funções do sistema. O arquivo principal é `src/sistema.py`, responsável por coordenar a execução.

A estrutura principal utilizada foi:

```
global-solutions-1.1/
├── README.md
├── data/
│   └── telemetria.json
└── src/
    ├── sistema.py
    └── modulos/
        ├── alertas.py
        ├── diagnostico.py
        ├── gerador_dados.py
        ├── modelos.py
        ├── previsao.py
        ├── recomendacoes.py
        └── telemetria.py
```

Cada arquivo possui uma responsabilidade específica. O `sistema.py` não concentra toda a lógica, mas chama as classes dos módulos auxiliares para executar o fluxo completo.

4. Como executar o sistema

Para executar o projeto, deve-se abrir o terminal na pasta do repositório e rodar:

```
python -m src.sistema
```

Caso o arquivo `data/telemetria.json` ainda não exista, o próprio sistema cria automaticamente uma base de dados simulada com 1200 leituras horárias. Isso evita que o programa dependa de uma base já pronta para funcionar.

Depois de carregar ou gerar os dados, o sistema executa o fluxo completo de análise e imprime um relatório final no terminal.

```
# Execute esta célula com o notebook dentro da pasta do projeto.  
# Ela chama o sistema da mesma forma indicada no README.  
!python -m src.sistema
```

5. Fluxo de funcionamento

O funcionamento do sistema segue uma sequência bem definida:

1. Verifica se existe o arquivo `data/telemetria.json` ;
2. Se o arquivo não existir, gera dados simulados de telemetria;
3. Carrega as leituras horárias da missão;
4. Organiza os dados em estruturas como listas, fila, pilha, dicionário, hierarquia e matriz;
5. Classifica o status operacional da missão;
6. Detecta inconsistências nos dados;
7. Gera alertas automáticos;
8. Aplica previsão por média móvel simples;
9. Gera recomendações operacionais;
10. Exibe o relatório final no terminal.

Esse fluxo torna o sistema mais próximo de uma central de monitoramento, pois ele recebe dados, interpreta a situação, identifica riscos e sugere ações.

6. Geração dos dados simulados

A classe `GeradorTelemetria` é responsável por criar a base de dados da missão. Ela gera leituras horárias contendo informações de energia, ambiente, módulos, subsistemas e eventos.

A geração foi feita com uma semente fixa (`seed=42`), o que ajuda a manter os resultados previsíveis durante os testes. O sistema cria dados como:

- geração de energia;
- consumo de energia;
- percentual de bateria;
- temperatura;
- radiação;
- qualidade de comunicação;
- velocidade do vento;
- status do sensor de oxigênio;
- status dos módulos críticos;
- eventos operacionais.

A bateria é atualizada ao longo do tempo conforme a diferença entre geração e consumo. Quando a geração é maior que o consumo, a tendência é a bateria subir. Quando o consumo é maior, a bateria cai.

7. Dados monitorados pela missão

Cada leitura horária representa o estado da missão em um determinado momento. Os dados principais são organizados em grupos:

Módulos críticos

Os módulos usam valores binários, em que **1** representa funcionamento ativo e **0** representa falha ou indisponibilidade.

Os principais módulos são:

```
suporte_vida
energia
comunicacao
habitat
laboratorio
armazenamento
```

Energia

Os dados de energia registram geração, consumo e bateria:

```
geracao_kwh
consumo_kwh
bateria_percentual
```

Ambiente

Os dados ambientais indicam condições externas ou internas relevantes para a missão:

```
temperatura_c
radiacao_msv
qualidade_comunicacao
velocidade_vento_kmh
sensor_oxigenio
```

Eventos

O sistema também registra eventos como falhas, alertas, reinicializações, mudanças de prioridade, modo econômico e inconsistências.

8. Estruturas de dados utilizadas

O projeto usa diferentes estruturas de dados para organizar as informações da missão.

Estrutura	Uso no sistema
Lista	Armazena séries de consumo, geração, temperatura e leituras horárias
Fila	Guarda alertas pendentes para serem processados por ordem de chegada
Pilha	Registra os últimos eventos críticos encontrados
Dicionário	Permite consultar rapidamente os módulos pelo nome
Hierarquia	Organiza subsistemas da missão, como energia, habitat e pesquisa
Matriz	Representa leituras energéticas no formato <code>[geração, consumo, bateria]</code>

Essa escolha deixa os dados mais fáceis de acessar e permite demonstrar a aplicação prática dos conteúdos estudados.

9. Organização da telemetria

A classe `TelemetriaMissao` é responsável por carregar o arquivo JSON e transformar os dados em estruturas internas usadas pelo restante do sistema.

Durante o carregamento, a classe separa as informações em listas específicas, como consumo energético, geração energética e temperatura. Ela também monta a matriz de energia, registra eventos no log, guarda eventos críticos em pilha e armazena os módulos em um dicionário.

Essa etapa é importante porque o sistema não trabalha diretamente com o JSON bruto o tempo todo. Primeiro os dados são lidos, depois são organizados, e só então as outras partes do programa fazem diagnóstico, previsão e recomendações.

10. Diagnóstico operacional

O diagnóstico operacional é feito pela classe `DiagnosticoOperacional`. Ela analisa a última leitura da telemetria e classifica o estado da missão como **NORMAL**, **ALERTA** ou **CRÍTICO**.

A regra principal de estado crítico é:

```
CRITICO =  
(bateria < 20 AND comunicacao == 0)  
OR  
(suporte_vida == 0)  
OR  
(radiacao > 90)
```

Essa regra foi pensada de forma conservadora. Se a bateria estiver muito baixa junto com falha de comunicação, se o suporte à vida falhar ou se a radiação estiver em nível perigoso, a missão entra em estado crítico.

Também existe uma classificação intermediária de alerta, usada quando a bateria está baixa, a radiação está elevada ou o sensor de oxigênio apresenta problema.

11. Inconsistência proposital

O sistema inclui uma inconsistência proposital nos dados para testar a capacidade de diagnóstico. Em determinado momento, o suporte à vida aparece como ativo, mas o sensor de oxigênio aparece desligado.

Essa situação é tratada como uma inconsistência porque os dois dados não combinam totalmente. Se o suporte à vida está ativo, espera-se que o sensor de oxigênio esteja funcionando ou pelo menos retornando uma leitura válida.

A detecção desse caso é importante porque, em sistemas críticos, nem todo problema aparece como uma falha direta. Às vezes o risco está em dados conflitantes, que podem levar a uma decisão errada se forem aceitos sem verificação.

12. Central de alertas

A geração de alertas é feita pela classe `CentralAlertas`. Ela analisa a última leitura da missão e cria alertas de acordo com situações de risco.

O sistema pode gerar alertas para casos como:

- falha no suporte à vida;
- comunicação instável ou indisponível;
- bateria abaixo de 20%;
- bateria abaixo de 40%;
- radiação perigosa;
- inconsistências nos sensores.

Cada alerta possui severidade, mensagem, recomendação e timestamp. Isso deixa a saída mais organizada e facilita a interpretação por quem está monitorando a missão.

13. Previsão por média móvel simples

A previsão foi implementada com a classe `PrevisorMediaMovel`. A técnica usada é a **média móvel simples**, com uma janela de 3 valores.

A fórmula usada é:

```
previsao = (valor_1 + valor_2 + valor_3) / 3
```

No sistema, essa previsão é aplicada sobre os últimos valores de consumo energético e geração energética. O objetivo não é criar um modelo avançado, mas sim estimar a tendência imediata da missão de forma simples e compreensível.

Essa previsão influencia as recomendações. Por exemplo, se o consumo previsto for maior que a geração prevista, o sistema pode recomendar ativar modo econômico ou desligar temporariamente sistemas não essenciais.

14. Recomendações operacionais

As recomendações são geradas pela classe `RecomendadorOperacional`. Ela recebe o status da missão, a última leitura, o consumo previsto e a geração prevista.

As recomendações são criadas de acordo com as condições encontradas. Alguns exemplos são:

- acionar protocolo crítico da missão;
- ativar modo economia;
- desligar temporariamente o laboratório;
- ativar comunicação de emergência;
- suspender atividades externas;
- priorizar suporte à vida;
- auditar sensores de oxigênio;
- manter operação normal.

Essa parte é importante porque o sistema não apenas aponta problemas. Ele também sugere ações para responder a esses problemas.

15. Modelagem matemática usada no projeto

A modelagem matemática principal aparece em duas partes do sistema: no comportamento da bateria e na previsão por média móvel.

15.1 Atualização da bateria

A bateria varia conforme a diferença entre geração e consumo:

```
bateria_nova = bateria_atual + ((geracao - consumo) / 18)
```

Se a geração for maior que o consumo, o resultado tende a aumentar a bateria. Se o consumo for maior que a geração, o resultado tende a diminuir a bateria.

Também existe um limite mínimo e máximo para a bateria, evitando valores irreais:


```
5% ≤ bateria ≤ 100%
```

15.2 Média móvel

A previsão por média móvel usa os últimos três valores registrados:

```
media_movel = soma_dos_ultimos_3_valores / 3
```

Essa técnica permite suavizar oscilações e estimar o comportamento do próximo ciclo de forma simples.

16. Exemplo de interpretação do sistema

Um exemplo de situação crítica seria:

```
bateria = 18%  
comunicacao = 0  
suporte_vida = 1  
radiacao = 55 mSv
```

Nesse caso, mesmo que o suporte à vida esteja funcionando e a radiação não esteja extrema, a missão pode ser classificada como **CRÍTICA**, pois a bateria está abaixo de 20% e a comunicação está indisponível.

A recomendação esperada seria reduzir consumo, ativar comunicação de emergência e priorizar módulos essenciais. Isso mostra como a lógica booleana ajuda o sistema a tomar decisões com base em combinações de condições.

17. Resultado exibido no terminal

Ao final da execução, o sistema imprime um relatório com as principais informações analisadas:

- nome da missão;
- quantidade de leituras analisadas;
- última leitura processada;
- status da missão;
- oito eventos selecionados do log;
- previsão de consumo e geração;
- inconsistências detectadas;
- alertas gerados;
- recomendações operacionais.

Essa saída resume o estado da missão e pode ser usada como base para apresentação do vídeo.

18. Pontos positivos da solução

A solução apresenta alguns pontos fortes:

- organização modular do código;
- geração automática de dados simulados;
- uso de estruturas de dados variadas;
- diagnóstico com lógica booleana;
- detecção de inconsistência proposital;
- alertas automáticos com recomendações;
- previsão simples por média móvel;
- relatório final claro no terminal.

A estrutura orientada a objetos também facilita melhorias futuras, já que cada classe tem uma responsabilidade bem definida.

19. Possíveis melhorias futuras

Mesmo funcionando para o objetivo da atividade, o sistema poderia ser expandido em versões futuras.

Algumas melhorias possíveis seriam:

- criar interface gráfica ou painel web;
- permitir filtros por período de tempo;
- salvar o relatório final em arquivo;
- gerar gráficos de bateria, consumo e geração;
- incluir mais sensores e módulos;
- comparar previsões diferentes;
- criar níveis de prioridade para as recomendações;
- adicionar testes automatizados.

Essas melhorias não são obrigatórias para o funcionamento atual, mas mostram caminhos possíveis para evoluir o projeto.

20. Conclusão

O sistema desenvolvido cumpre a proposta de monitorar uma missão espacial experimental usando dados simulados de telemetria. Ele organiza as informações em estruturas de dados, interpreta o estado dos módulos, identifica riscos, detecta inconsistências e gera recomendações operacionais.

A divisão em classes tornou o projeto mais organizado e facilitou a separação das responsabilidades. A classe de telemetria cuida dos dados, o diagnóstico classifica a missão, a central de alertas registra problemas, o previsor calcula tendências e o recomendador sugere ações.

De forma geral, o projeto mostra como conceitos de programação, lógica booleana, estruturas de dados e modelagem matemática podem ser aplicados em um cenário de monitoramento espacial. A solução não se limita a processar dados: ela transforma as leituras em diagnósticos e recomendações úteis para manter a missão segura.

21. Referências do projeto

- Repositório do projeto: <https://github.com/FIAP-workd/global-solutions-1.1>
- Arquivo principal: `src/sistema.py`
- Módulos auxiliares: `src/modulos/`
- Base de dados simulada: `data/telemetria.json`

